to MVC or that should only apply to certain routes.

Both of these can apply in exception handling. Exception filters are part of the MVC framework, so they have access to the context in which the error occurred, such as the action or Razor Page that was executing. This can be useful for logging additional details when errors occur, such as the action parameters that caused the error.

WARNING If you use exception filters to record action method arguments, make sure you're not storing sensitive data in your logs, such as passwords or credit card details.

You can also use exception filters to handle errors from different routes in different ways. Imagine you have both Razor Pages and web API controllers in your app, as we do in the recipe app. What happens when an exception is thrown by a Razor Page?

As you saw in chapter 4, the exception travels back up the middleware pipeline and is caught by exception handler middleware. The exception handler middleware reexecutes the pipeline and generates an HTML error page.

That's great for your Razor Pages, but what about exceptions in your web API controllers? If your API throws an exception and consequently returns HTML generated by the exception handler middleware, that's going to break a client that called the API expecting a JavaScript Object Notation (JSON) response!

TIP The added complexity introduced by having to handle these two very different clients is the reason I prefer to create separate applications for APIs and server-rendered apps.